

SOFTWARE TEST PLAN

Project: PrepMate AI

Document Type: Software Test Plan

Testing Type: Manual Testing

Application Type: Web Application

Version: 1.0

Prepared By: Pranali Santosh Pawar

1. Introduction

PrepMate AI is an AI-powered web application designed to help users prepare for job interviews and improve their resumes. The application allows users to create an interview preparation plan by providing their resume, job description, and self-description.

The system analyzes the provided information using AI and generates an interview preparation report containing technical questions, behavioral questions, skill gaps, and a preparation roadmap. Users can also generate and download an optimized resume based on their interview report.

This Test Plan defines the testing approach, scope, objectives, resources, test types, and activities required to validate the quality and reliability of the PrepMate AI application.

2. Testing Objectives

The primary objectives of testing are:

1. Verify that all major application functionalities work according to their expected behavior.
2. Ensure that users can successfully register, log in, use the application, and log out.
3. Validate resume upload and input handling.
4. Verify that interview reports are generated correctly from valid user inputs.
5. Verify that generated interview reports display the expected information.
6. Validate resume generation and download functionality.
7. Identify defects, unexpected behavior, validation issues, and edge cases.
8. Verify that defects are correctly reproduced, documented, fixed, and retested.
9. Ensure that changes to one functionality do not negatively affect existing functionality.
10. Evaluate basic usability and compatibility across supported browsers and screen sizes.

3. Scope of Testing

3.1 In Scope

The following application functionalities are included in testing:

A. User Registration

- Registration with valid details
- Required-field validation
- Duplicate username/email handling
- Invalid input handling
- Successful account creation

B. User Login

- Login with valid credentials
- Login with invalid credentials
- Empty-field validation
- Authentication/session handling
- Unauthorized access handling

C. User Logout

- Successful logout
- Session/token invalidation
- Accessing protected functionality after logout

D. Interview Plan / Input Form

- Job description input
- Self-description input
- Resume upload
- Required-field validation
- File type validation
- File size validation
- Valid and invalid input combinations

E. AI Interview Report Generation

- Report generation with valid inputs
- Handling incomplete inputs
- AI/API failure scenarios
- Report creation and storage
- Loading/error states

F. Interview Report

- Report retrieval
- Technical questions
- Behavioral questions
- Skill gaps
- Preparation plan/roadmap
- Navigation between report sections
- Report data consistency

G. Resume Generation

- Resume generation from an interview report
- PDF generation
- PDF download
- File naming and format
- Invalid/non-existing report handling

H. History / Previous Reports

- Retrieving previous interview reports
- Correct report association with logged-in user
- Opening a selected report
- Empty-history handling

I. UI and Compatibility

- Layout and navigation
- Form usability
- Input validation messages
- Basic responsive behavior

- Browser compatibility
 - Buttons, links, and interactive elements
-

3.2 Out of Scope

The following areas are outside the primary scope of this manual testing phase:

- Performance/load testing
- Stress testing
- Full penetration/security testing
- Source-code-level unit testing
- AI model training and internal model accuracy
- Third-party AI provider infrastructure
- Production server infrastructure
- Database performance benchmarking

Basic security-related functional scenarios such as authentication and unauthorized access will still be tested.

4. Application Modules

Testing will be divided into the following modules:

Module ID	Module
M01	User Registration
M02	User Login
M03	Authentication & Session
M04	Logout
M05	Interview Plan/Input
M06	Resume Upload
M07	AI Interview Analysis
M08	Interview Report
M09	Previous Reports/History
M10	Resume Generation

M11	Error Handling & Validation
M12	UI & Browser Compatibility

5. Test Types

The following testing techniques will be performed.

5.1 Functional Testing

Verify that application functions behave according to their expected requirements.

Examples:

- User registration
- Login
- Resume upload
- Report generation
- Resume download

5.2 Smoke Testing

Perform a basic set of tests to determine whether the application build is stable enough for detailed testing.

Examples:

- Application launches
- User can register/login
- Interview plan can be created
- Report can be generated
- Logout works

5.3 Regression Testing

Re-execute previously passed test cases after changes or bug fixes to ensure existing functionality has not been affected.

5.4 Retesting

Test previously failed test cases after the corresponding defect has been fixed.

5.5 Negative Testing

Verify that the system handles invalid or unexpected inputs correctly.

Examples:

- Invalid login credentials
- Empty form fields
- Unsupported resume file
- Oversized file
- Invalid report ID

5.6 Boundary Value Testing

Test values at and around defined limits.

Examples:

- Resume file size near the maximum allowed size
- Minimum/maximum input lengths
- Empty input
- Very large input

5.7 Equivalence Partitioning

Divide input data into valid and invalid groups and select representative values from each group.

Example:

Resume file:

- Valid PDF
- Valid DOCX
- Unsupported file
- Corrupted file
- Oversized file

5.8 Usability Testing

Verify that users can understand and operate the application's interface easily.

5.9 Compatibility Testing

Verify basic functionality and layout across supported browsers and different screen sizes.

6. Test Approach

Testing will primarily follow a **manual black-box testing approach**.

Test cases will be derived from:

- Application functionality
- User workflows
- Expected behavior
- Input validation rules
- API behavior
- Acceptance criteria
- Positive and negative scenarios
- Boundary conditions

The tester will execute test cases using predefined test data and record the actual result.

If the actual result differs from the expected result, a defect will be documented.

The defect will then follow this lifecycle:

New → Assigned → Fixed → Retest → Verified/Closed

If a defect remains unresolved during retesting, it will be marked as **Reopened**.

7. Test Environment

Frontend

- React.js
- Vite
- HTML/CSS/SCSS
- JavaScript

Backend

- Node.js
- Express.js

Database

- MongoDB

API

- REST APIs

Testing Tools

- Browser Developer Tools
- Postman
- Jira / Issue Tracking Tool
- Test Case Spreadsheet
- GitHub

Browsers

Testing will initially be performed on:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

Operating System

- Windows
-

8. Test Data

Test data will include:

User Data

- Valid username
- Valid email
- Invalid email
- Duplicate username
- Duplicate email
- Valid password
- Invalid password
- Empty credentials

Resume Data

- Valid PDF resume
- Valid DOCX resume, if supported by the application
- Unsupported file format
- Corrupted file

- Empty file
- File near the maximum size
- Oversized file

Job Description

- Valid job description
- Empty job description
- Short job description
- Long job description
- Special characters
- Multiple-line job description

Self Description

- Valid self-description
- Empty self-description
- Short input
- Long input
- Special characters

9. Entry Criteria

Testing will begin when:

1. The application can be successfully launched.
2. Required frontend and backend services are available.
3. Database connectivity is established.
4. Required test environment is configured.
5. Test data is available.
6. Major application functionality is accessible for testing.

10. Exit Criteria

Testing will be considered complete when:

1. All planned test cases have been executed.

2. Critical and high-severity defects have been resolved or formally accepted.
 3. Failed test cases have been retested after fixes.
 4. Regression testing has been completed.
 5. Test execution results have been documented.
 6. Major application workflows have been successfully validated.
 7. A final test summary has been prepared.
-

11. Defect Management

All identified defects will be documented with sufficient information for reproduction.

Each defect report should contain:

- Defect ID
- Defect title
- Module
- Description
- Preconditions
- Steps to reproduce
- Test data
- Expected result
- Actual result
- Severity
- Priority
- Environment
- Screenshot/evidence
- Defect status

Defect Lifecycle

New → Assigned → In Progress → Fixed → Retest → Closed

If the issue is still present after the fix:

Retest → Reopened

12. Severity Classification

Severity	Description
Critical	Application crash, data loss, major security issue, or complete failure of a critical workflow
High	Major functionality is unavailable or significantly affected
Medium	Functionality works partially or has a significant usability/functional issue
Low	Minor UI, cosmetic, or low-impact issue

13. Priority Classification

Priority	Description
P1 – High	Must be fixed immediately before release
P2 – Medium	Should be fixed before release if possible
P3 – Low	Can be addressed in a future release

Severity represents **impact**, while priority represents **how urgently the defect should be fixed**.

14. Risks and Mitigation

Risk	Impact	Mitigation
AI API unavailable	Report generation cannot be tested	Test error handling and retry behavior
Invalid/large resume file	Upload may fail unexpectedly	Perform file validation and boundary testing
Database connectivity failure	User/report data may not load	Test appropriate error handling
Authentication/session issues	Protected functionality may be inaccessible	Perform authentication and session testing
Browser compatibility issues	UI/functionality may behave differently	Perform cross-browser testing
Changes during testing	Existing functionality may break	Perform regression testing

15. Test Deliverables

The following QA artifacts will be maintained in the GitHub repository:

1. **Test Plan**
 2. **Test Scenarios**
 3. **Test Cases**
 4. **Test Execution Report**
 5. **Defect/Bug Reports**
 6. **Retesting Report**
 7. **Regression Testing Report**
 8. **Test Summary Report**
 9. **Screenshots and supporting evidence**
 10. **README documenting the QA process**
-

16. Traceability

Each major application requirement/module will be mapped to corresponding test scenarios and test cases.

Example:

User Login → Test Scenarios → Test Cases → Execution Result → Defect (if any) → Retest Result

This ensures that important functionality is covered and that testing can be traced from requirement to final result.

17. Overall Testing Strategy

The testing process for PrepMate AI will follow the following sequence:

1. **Requirement/Feature Understanding**
2. **Test Scenario Identification**
3. **Test Case Design**
4. **Test Data Preparation**
5. **Smoke Testing**
6. **Functional Testing**
7. **Negative & Boundary Testing**
8. **Defect Reporting**
9. **Defect Retesting**

10. Regression Testing

11. Test Summary & Release Readiness

18. Approval

Role	Name	Status
QA Tester	Pranali Santosh Pawar	Prepared
Project/Development Team	—	Pending
Reviewer	—	Pending

End of Test Plan